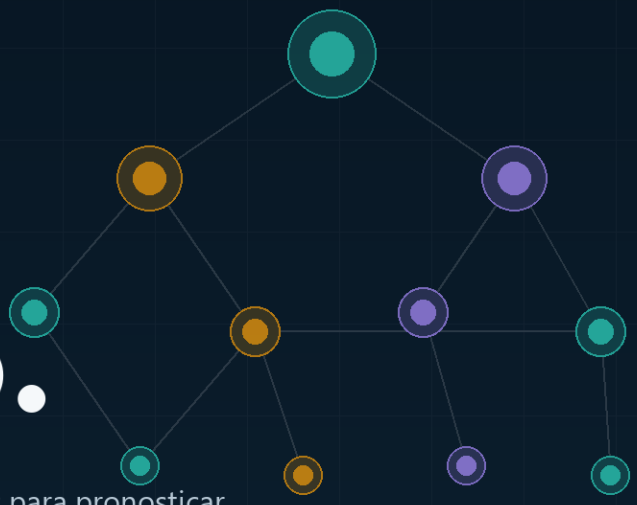
BOSQUE DE DECISIONES

TRABAJO FINAL · APRENDIZAJE SUPERVISADO

Aprender *dividiendo* el espacio.



Tres ensambles, tres decisiones de negocio: Extra Trees para pronosticar demanda, AdaBoost para cribar diabetes y CatBoost para anticipar abandono universitario. Teoría, entrenamiento real y simulación en el navegador.

EXTRA TREES

RMSE 1.998

R^2 0.607 · Walmart M5

modelo entrenado · métricas de test

ADABOOST

ROC-AUC 0.821

F1 0.758 · CDC BRFSS

modelo entrenado · métricas de test

CATBOOST

MACRO-F1 0.693

AUC 0.886 · UCI Dropout

modelo entrenado · métricas de test

Luchi-Marita · Equipo 2

Aprendizaje Supervisado · Familia 1: Árboles y Ensambles

Octubre de 2026

Python · scikit-learn · CatBoost

0. Resumen ejecutivo

Este informe documenta el diseño, el entrenamiento y la evaluación de tres ensambles de árboles aplicados a tres problemas reales con estructuras estadísticas distintas: pronóstico de ventas diarias en retail (Walmart M5), cribado de diabetes en salud pública (CDC BRFSS 2015) y predicción de abandono universitario (UCI Student Dropout). El objetivo no es declarar un ganador absoluto, sino mostrar **cuándo y por qué** cada familia de ensambles es la elección correcta, con evidencia cuantitativa y una simulación interactiva que permite auditar el comportamiento del modelo en el navegador.

Extra Trees se evaluó sobre una muestra estratificada de 800 series de producto-tienda (400 000 filas, 32 variables) con un split temporal de 355 200 / 22 400 / 22 400 filas. El modelo final (366 árboles, profundidad 36, 70 % de variables por split) obtuvo RMSE = 1.998 unidades/día, MAE = 0.990, R^2 = 0.607 y WAPE = 70.2 %. Frente al baseline estacional `lag_7` (RMSE 2.776), la reducción de error es de 28.0%.

AdaBoost se entrenó sobre 69 057 encuestas balanceadas de BRFSS (25 variables). La búsqueda seleccionó 370 estimadores débiles de profundidad 3, con learning rate 0.10. En test, ROC-AUC = 0.821 y F1 = 0.758 con umbral 0.50. El umbral operativo 0.38, elegido en una partición interna de entrenamiento, eleva el recall a 0.912 y el F1 a 0.773, a costa de una precisión de 0.670. El Brier score es 0.178, lo que exige recalibración antes de comunicar probabilidades individuales.

CatBoost se aplicó a 4 424 estudiantes con 39 variables (18 categóricas nativas). El modelo final usó 172 árboles tras early stopping, profundidad 5 y pesos de clase balanceados. En test alcanzó accuracy = 0.731, balanced accuracy = 0.705, macro-F1 = 0.693 y AUC-OvR = 0.886. La clase Enrolled es la más difícil (F1 = 0.503), mientras Dropout y Graduate superan 0.74 de F1.

ENTREGABLES

Repositorio con código y datos descargables: https://github.com/Luchi-Marita/arboles-ensambles-equipo2. Dashboard interactivo: https://luchi-marita.github.io/arboles-ensambles-equipo2/. Informe en Word y PDF, figuras, tablas y modelos exportados a JSON con verificación de paridad de inferencia (diferencia máxima < 5×10^{-16}).

Palabras clave: Extra Trees, AdaBoost, CatBoost, boosting, bagging, demanda intermitente, cribado, abandono universitario, calibración, interpretabilidad.

·. Contenido

0. Resumen ejecutivo 2
·. Contenido 3
1. Introducción 4
1.1 Contexto y motivación 4
1.2 Objetivos 4
1.3 Preguntas de investigación 4
1.4 Aportes y organización 4
2. Fundamentos matemáticos 5
2.1 Árboles de decisión y particiones recursivas 5
2.2 Extra Trees: aleatoriedad extrema y promediado 5
2.3 AdaBoost: boosting adaptativo y pérdida exponencial 6
2.4 CatBoost: boosting con categóricas nativas 7
2.5 Métricas y validación 8
3. Datos y metodología 9
3.1 Conjuntos de datos 9
3.2 Protocolo de entrenamiento y búsqueda 10
3.3 Reproducibilidad y exportación web 10
4. Resultados 11
4.1 Extra Trees · Walmart M5 11
4.2 AdaBoost · CDC BRFSS 14
4.3 CatBoost · UCI Student Dropout 17
4.4 Comparación entre los tres ensambles 20
5. Aplicaciones y análisis crítico 22
5.1 Retail: inventario con demanda intermitente 22
5.2 Salud pública: cribado con recall alto 22
5.3 Educación: alerta temprana de abandono 22
5.4 Guía de selección 23
6. Conclusiones 24
6.1 Trabajo futuro 24
7. Referencias 25
A. Anexo A. Grillas de hiperparámetros 26
B. Anexo B. Reproducibilidad y estructura 27
C. Anexo C. Declaración de uso responsable 28

1. Introducción

1.1 Contexto y motivación

Los árboles de decisión son atractivos porque aproximan funciones no lineales sin imponer una forma paramétrica, toleran escalas heterogéneas y producen reglas auditables. Su debilidad es la varianza: un árbol profundo cambia mucho ante pequeñas perturbaciones de los datos. Los ensambles atacan ese problema por dos caminos. El **bagging** promedia muchos árboles entrenados sobre muestras o perturbaciones independientes y reduce la varianza. El **boosting** construye árboles de forma secuencial, cada uno corrigiendo los errores del anterior, y reduce el sesgo. Extra Trees, AdaBoost y CatBoost son tres expresiones maduras de esas ideas, con supuestos y costos distintos.

La elección de los tres casos no es accidental. M5 representa un problema de regresión con demanda intermitente, muchos ceros, estacionalidad semanal y precios; BRFSS es un problema binario con variables clínicas y de estilo de vida, donde el costo de un falso negativo es alto; UCI Dropout es un problema multiclase con categorías nominales de alta cardinalidad (curso, modo de admisión, cualificación de los padres) y una clase intermedia ambigua. Cada estructura exige una estrategia de validación y una métrica diferente.

1.2 Objetivos

- **Objetivo general:** implementar, ajustar y comparar Extra Trees, AdaBoost y CatBoost en tres casos de uso reales, con un pipeline reproducible y una interfaz web que permita reproducir el entrenamiento y simular inferencias.
- **Objetivos específicos:** (i) documentar la formulación matemática y los supuestos de cada ensamble; (ii) construir splits sin fuga de información y justificar las métricas; (iii) ajustar hiperparámetros con validación cruzada y reportar intervalos de confianza; (iv) analizar críticamente el valor y los límites de cada modelo en su dominio; (v) publicar código, artefactos y una simulación interactiva.

1.3 Preguntas de investigación

1. ¿Qué compromiso entre sesgo, varianza e interpretabilidad ofrece cada ensamble en un régimen de datos distinto?
2. ¿Cómo cambia la decisión operativa cuando se elige un umbral o una métrica en lugar de otra?
3. ¿Qué evidencia se necesita para pasar de un buen resultado offline a una recomendación responsable en retail, salud o educación?

1.4 Aportes y organización

El trabajo aporta (i) un pipeline reproducible con semillas fijas y separación estricta de train, validación y test; (ii) una comparación cuantitativa con métricas adecuadas a cada tarea; (iii) una exportación de los modelos a JavaScript con verificación numérica de paridad; y (iv) un análisis crítico de aplicaciones que explicita riesgos, sesgos y condiciones de uso. El resto del informe se organiza así: la Sección 2 presenta los fundamentos matemáticos; la Sección 3 describe datos y metodología; la Sección 4 reporta resultados; la Sección 5 analiza aplicaciones y límites; la Sección 6 concluye; los anexos contienen grillas, comandos y estructura del repositorio.

2. Fundamentos matemáticos

2.1 Árboles de decisión y particiones recursivas

Un árbol de regresión divide el espacio de entrada en regiones disjuntas R_1, …, R_M mediante reglas binarias del tipo x_j ≤ s. En cada región, la predicción es una constante c_m, normalmente la media de las observaciones que caen en ella:

$$\hat{y}(x) = \sum_{m=1}^{M} c_m \mathbf{1}\{x \in R_m\} \tag{1}$$

La partición se elige de forma greedy: en cada nodo se evalúan candidatos (variable, umbral) y se maximiza la reducción de impureza. Para regresión, la impureza es la varianza o el error cuadrático medio; para clasificación, el índice Gini o la entropía. La ganancia de un split s es:

$$\Delta I(s) = I(R) - \frac{|R_L|}{|R|} I(R_L) - \frac{|R_R|}{|R|} I(R_R) \tag{2}$$

donde R_L y R_R son los nodos hijos. El árbol crece hasta una profundidad máxima, un tamaño mínimo de hoja o una ganancia mínima. Sin poda, un árbol profundo tiene bajo sesgo y alta varianza: es un estimador inestable. Los ensambles estabilizan esa estimación promediando o corrigiendo árboles.

La descomposición sesgo-varianza del error cuadrático esperado es $E[(y - \hat{f}(x))^2] = \sigma^2 + (f(x) - E[\hat{f}(x)])^2 + \mathrm{Var}(\hat{f}(x))$. El bagging reduce el tercer término al promediar modelos aproximadamente independientes; el boosting reduce el segundo al ajustar residuales de forma secuencial. Ninguno elimina el ruido irreducible σ^2 ni corrige por sí solo un sesgo de selección en los datos.

2.2 Extra Trees: aleatoriedad extrema y promediado

Extra Trees (Extremely Randomized Trees) es un ensamble de tipo bagging con dos diferencias respecto de Random Forest. Primero, no remuestrea con reemplazo: cada árbol se entrena con el conjunto completo (o una muestra sin reemplazo, según la implementación). Segundo, en cada nodo no busca el mejor umbral entre todos los candidatos: sortea varios umbrales por variable y elige el mejor del sorteo. Esa aleatoriedad adicional reduce la correlación entre árboles y suele disminuir la varianza, a costa de un ligero aumento del sesgo.

La predicción de regresión es el promedio de M árboles independientes:

$$\hat{y}_{\mathrm{ET}}(x) = \frac{1}{M} \sum_{m=1}^{M} T_m(x) \tag{3}$$

La aleatoriedad de los umbrales actúa como regularización: impide que todos los árboles exploten el mismo punto de corte óptimo y hace que el ensamble sea menos sensible a ruido y outliers. En scikit-learn, `ExtraTreesRegressor` expone `max_features`, `min_samples_leaf`, `min_samples_split`,

`max_depth` y `n_estimators`. La complejidad de entrenamiento es O(M · n log n) en el caso balanceado y el método es paralelizable por árbol.

POR QUÉ ENCAJA EN M5

La demanda diaria tiene una masa grande de ceros, picos y una estacionalidad semanal fuerte. Un modelo lineal subestima la no linealidad; un árbol único sobreajusta. Extra Trees promedia muchos árboles poco correlacionados y tolera los ceros sin transformaciones especiales. El precio es que suaviza picos extremos y no extrapola tendencias fuera del rango observado.

2.3 AdaBoost: boosting adaptativo y pérdida exponencial

AdaBoost combina clasificadores débiles h_t (típicamente árboles poco profundos) con pesos α_t. En cada ronda t, cada observación i tiene un peso w_i^{(t)}. El clasificador débil minimiza el error ponderado ε_t; su peso en el ensamble crece cuando ε_t es pequeño:

$$\varepsilon_t = \sum_i w_i^{(t)} \mathbf{1}[h_t(x_i) \neq y_i], \qquad \alpha_t = \tfrac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t} \tag{4}$$

Después de entrenar h_t, los pesos se actualizan para castigar los errores y se normalizan por Z_t para que sumen uno:

$$w_i^{(t+1)} = \frac{w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \tag{5}$$

La predicción final es el signo del voto ponderado:

$$\hat{y}(x) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x) \right) \tag{6}$$

AdaBoost puede verse como un descenso por coordenadas en el espacio de funciones que minimiza la pérdida exponencial L(y, f) = exp(−y f(x)). Si cada clasificador débil tiene margen γ, es decir, ε_t ≤ ½ − γ, el error de entrenamiento está acotado por exp(−2Tγ²). Esa cota explica por qué añadir rondas puede seguir mejorando el ajuste, pero también por qué el método es sensible a etiquetas ruidosas: un ejemplo mal etiquetado recibe pesos crecientes y puede dominar una ronda.

En scikit-learn, `AdaBoostClassifier` con `algorithm='SAMME'` (el usado aquí) combina los votos y convierte la función de decisión en probabilidad mediante una sigmoide calibrada de forma aproximada. En la exportación a JavaScript se replica exactamente esa transformación: p = σ(2 · Σ α_t sign(h_t) / Σ α_t). Por eso las probabilidades deben interpretarse como scores de riesgo, no como probabilidades clínicas calibradas.

ELECCIÓN DEL UMBRAL

El umbral 0.50 maximiza exactitud, pero en cribado el costo de un falso negativo es mayor. Se eligió 0.38 sobre una partición interna del 15 % del entrenamiento, nunca sobre test. El resultado es un clasificador con recall 0.912 y precisión 0.670: apto para priorizar derivaciones, no para diagnosticar.

2.4 CatBoost: boosting con categóricas nativas

CatBoost es un gradient boosting que construye un modelo aditivo. Para clasificación multiclase con K clases, la predicción se obtiene con una softmax sobre las funciones F_k:

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x) \tag{7}$$

$$p_k(x) = \frac{\exp(F_k(x))}{\sum_{j=1}^{K} \exp(F_j(x))} \tag{8}$$

La pérdida es la entropía cruzada multiclase:

$$\mathcal{L} = -\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik} \ln p_{ik} \tag{9}$$

En cada iteración se ajusta un árbol a los gradientes g_i = p_i − y_i y a los hessianos h_i = p_i(1 − p_i). Cada hoja recibe el valor Newton que minimiza la aproximación cuadrática local:

$$w_j^{\star} = -\frac{\sum_{i \in R_j} g_i}{\sum_{i \in R_j} h_i + \lambda} \tag{10}$$

La innovación de CatBoost es el tratamiento de variables categóricas. En lugar de un one-hot de alta dimensión, reemplaza cada categoría por una estadística objetivo calculada con una permutación aleatoria y solo con ejemplos anteriores en esa permutación. Para la categoría k:

$$\hat{x}_{i,k} = \frac{\sum_{j:\,\sigma(j)<\sigma(i)} \mathbf{1}[x_{j,k} = x_{i,k}]\, y_j + a\, p}{\sum_{j:\,\sigma(j)<\sigma(i)} \mathbf{1}[x_{j,k} = x_{i,k}] + a} \tag{11}$$

donde a es un parámetro de suavizado y p es el prior. Esa restricción evita el target leakage del target encoding clásico y permite usar categorías de alta cardinalidad sin explosión de columnas. Los árboles son *oblivious*: el mismo split se aplica en todo un nivel, lo que reduce el sobreajuste, acelera la inferencia y

produce importancias estables. El boosting se controla con shrinkage ν, regularización L2 λ, profundidad y early stopping.

POR QUÉ ENCAJA EN UCI

El dataset tiene 18 variables categóricas, varias con decenas de niveles (curso, modo de admisión, cualificación de los padres). Un one-hot clásico diluye la señal y crea columnas casi vacías. CatBoost aprende una representación ordenada de cada categoría y comparte estadísticas entre niveles, lo que mejora la generalización con solo 4 424 filas.

2.5 Métricas y validación

En regresión se reportan RMSE, MAE, R^2 y WAPE. RMSE penaliza más los errores grandes y tiene las unidades de la variable objetivo; MAE es robusto y fácil de comunicar; R^2 compara con la media; WAPE (suma de errores absolutos / suma de valores reales) es útil cuando hay ceros y MAPE se vuelve inestable.

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}, \qquad \mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i| \tag{12}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{13}$$

En clasificación binaria se reportan accuracy, precisión, recall (sensibilidad), especificidad, F1, ROC-AUC, average precision, Brier score y log-loss. El F1 combina precisión y recall:

$$F_1 = 2 \cdot \frac{\mathrm{precision} \cdot \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}} \tag{14}$$

En multiclase se usa macro-F1 (promedio no ponderado por clase), balanced accuracy y AUC-OvR. Macro-F1 es exigente con las clases minoritarias: una clase con mal desempeño arrastra el promedio. Los intervalos de confianza se estiman con bootstrap percentil (2 000 remuestreos para regresión y 500–1 000 para clasificación), y el umbral de AdaBoost se selecciona en una partición interna, no en test. La validación de Extra Trees es temporal (últimos 28 días de validación y 28 de test por serie); la de AdaBoost y CatBoost es estratificada 80/20.

3. Datos y metodología

3.1 Conjuntos de datos

La Tabla 1 resume los tres datasets. Los tamaños son los efectivamente usados después de limpieza; no se reportan cifras nominales de los repositorios originales.

Caso	Fuente	Tarea	Filas	Variables	Clases / target
Extra Trees	Walmart M5	Regresión	400 000	32	ventas diarias (0–141)
AdaBoost	CDC BRFSS 2015	Binaria	69 057	25	diabetes / no diabetes
CatBoost	UCI Student Dropout	Multiclase	4 424	39	Dropout / Enrolled / Graduate

Tabla 1. Resumen de los datasets y de la tarea supervisada.

Walmart M5

M5 contiene 30 490 series de producto-tienda y un calendario de 1 969 días; la extracción de ventas usada abarca 1 913 días (2011-01-29 a 2016-04-24). Para un experimento docente reproducible se tomó una muestra estratificada de 80 series por tienda (800 series, 10 tiendas) y la ventana de 500 días comprendida entre 2014-12-12 y 2016-04-24. El target es la venta diaria en unidades. La muestra conserva la estructura jerárquica (tienda, departamento, categoría, estado), el calendario SNAP y los eventos, y el precio de venta semanal.

- **Split temporal:** 355 200 filas de entrenamiento, 22 400 de validación (28 días) y 22 400 de test (28 días). Nunca se mezclan fechas.
- **Features (32):** identificadores categóricos, calendario (día de semana, mes, día del año, fin de semana, SNAP, evento), precio y variaciones, lags 1/7/14/28/364, medias y desviaciones móviles 7/28/90, máximo móvil, ratio de ceros y estadísticas históricas item/tienda/departamento.
- **Fuga de información:** las estadísticas históricas se estiman solo con el período anterior a la ventana; los lags usan la historia completa previa y se desplazan por serie.
- **Distribución:** 59.75 % de ceros, media 1.31 y desviación 3.18 unidades/día; el máximo es 141. Es una distribución cero-inflada y muy asimétrica.

CDC BRFSS 2015

La versión balanceada del Indicador de Diabetes de BRFSS contiene 70 692 respuestas con 21 variables demográficas, clínicas y de estilo de vida. Tras eliminar 1 635 duplicados quedan 69 057 filas, con una tasa de positivos de 50.8 %. El target `Diabetes_binary` indica diagnóstico de diabetes o prediabetes. Se añadieron cuatro variables derivadas: comorbilidad (suma de hipertensión, colesterol alto, derrame y enfermedad cardíaca), hábitos saludables, categoría de IMC y días de salud mental+física. El split es estratificado 80/20 (13 812 filas de test).

- **Por qué no se usa el dataset completo de 253 680:** la versión balanceada facilita la comparación y evita que un clasificador trivial prediga la clase mayoritaria; se documenta como decisión de diseño.
- **Valores:** todas las variables son numéricas o binarias; no hay faltantes explícitos. El escalado se ajusta dentro del pipeline, solo con train.
- **Riesgo:** la encuesta es autodeclarada y el balanceo artificial no representa la prevalencia poblacional. Las probabilidades no deben interpretarse como riesgo clínico absoluto.

UCI Student Dropout

El dataset contiene 4 424 estudiantes y 36 variables originales (18 categóricas y 18 numéricas) sobre estado civil, modo de admisión, curso, cualificaciones y ocupaciones de los padres, edad, notas de admisión, unidades curriculares aprobadas/matriculadas y variables macroeconómicas. Se añadieron tres ratios derivados: aprobación en primer semestre, aprobación en segundo semestre y cambio de nota entre semestres, hasta 39 variables. Las clases son Dropout (1 421), Enrolled (794) y Graduate (2 209). El split estratificado 80/20 deja 885 estudiantes en test.

- **Categóricas nativas:** CatBoost recibe los 18 campos como categóricos, sin one-hot manual; el modelo demo del navegador usa one-hot para poder exportarse a JSON de forma compacta.
- **Clase ambigua:** Enrolled es una categoría intermedia entre Dropout y Graduate; su F1 cercano a 0.50 no es un fallo aislado, sino una consecuencia de la definición del target.
- **Privacidad:** el dataset es público y anonimizado, pero cualquier despliegue real debe evaluar reidentificación y consentimiento.

3.2 Protocolo de entrenamiento y búsqueda

Los tres experimentos usan semilla 42 y un pipeline de scikit-learn. La búsqueda de hiperparámetros se hizo con `RandomizedSearchCV` y validación cruzada. En Extra Trees, para contener el costo, se muestrearon 120 000 filas de entrenamiento y se usó una única partición explícita train/validación (sin fuga: el índice de validación queda fuera del de entrenamiento). En AdaBoost y CatBoost se usó `StratifiedKFold` de 3 folds. La Tabla 2 resume los espacios y el mejor punto encontrado.

Modelo	Búsqueda	Mejor configuración	Métrica CV
Extra Trees	12 configuraciones, 1 split temporal	boot=False · depth=36 · mf=0.7 · leaf=4 · split=2 · n=366	RMSE 2.044
AdaBoost	14 configuraciones, 3-fold estratificado	depth=3 · leaf=20 · lr=0.1 · n=370	AUC 0.822
CatBoost	14 configuraciones, 3-fold estratificado	rs=1 · lr=0.1 · l2=5 · iter=350 · depth=5 · bt=0 · weights=Balanced	macro-F1 0.726

Tabla 2. Espacios de búsqueda y mejor configuración. Los valores completos están en los CSV de resultados y en el Anexo A.

La selección no se hizo por el mejor score de test, sino por validación cruzada. El test se usó una sola vez, después de fijar hiperparámetros y umbral. En Extra Trees, el modelo final se reentrenó con train+validación (práctica estándar para la entrega) y se evaluó en el bloque temporal de test; en AdaBoost y CatBoost, el modelo final se entrenó con el 80 % de train y se evaluó en el 20 % reservado. El umbral de AdaBoost se eligió en un 15 % interno del train, separado del test.

3.3 Reproducibilidad y exportación web

Todo el pipeline se ejecuta con `python run_all.py`, que descarga los datos (si faltan), entrena los tres modelos, genera figuras y tablas y exporta los artefactos web. Las rutas y la semilla están centralizadas en `src/config.py`. Los modelos principales se guardan con joblib; el de Extra Trees pesa 2.9 GB y está excluido de Git, mientras que el modelo demo (44 árboles) se exporta a JSON de 7.4 MB para el navegador.

La exportación a JavaScript no es una caja negra: el script `src/export_web.py` implementa la misma lógica de inferencia en Python y en el JSON exportado, y compara predicciones en muestras de test. Las diferencias máximas absolutas son 4.4×10^{-16} para Extra Trees, 0.0 para AdaBoost y 5.6×10^{-17} para CatBoost, es decir, ruido de punto flotante. El dashboard permite reproducir las curvas de entrenamiento, ajustar perfiles y ver la contribución de cada árbol o variable sin depender de un servidor de modelos.

4. Resultados

Esta sección reporta los resultados en el conjunto de test y los contrasta con baselines sencillos. Las figuras se generaron a partir de los mismos artefactos que alimentan el dashboard; cualquier número del texto puede rastrearse hasta `artifacts/tables/*.json`.

4.1 Extra Trees · Walmart M5

Búsqueda y configuración

La mejor configuración fue **boot=False · depth=36 · mf=0.7 · leaf=4 · split=2 · n=366**, con RMSE de validación 2.044. La diferencia con la segunda configuración es pequeña (RMSE 2.045), lo que sugiere una meseta: el modelo no es extremadamente sensible a los hiperparámetros dentro de la región explorada. El modelo final reentrenado con train+validación obtuvo el rendimiento de test de la Tabla 3.

Métrica	Extra Trees (test)	Baseline lag_7	Mejora
RMSE (unidades/día)	1.998	2.776	28.0%
MAE (unidades/día)	0.990	1.245	20.5%
R^2	0.607	0.241	—
WAPE (%)	70.2	88.3	—
Sesgo medio	-0.049	0.021	—

Tabla 3. Métricas de Extra Trees en test frente al baseline estacional. La mejora se calcula sobre RMSE y MAE.

El intervalo bootstrap del RMSE es [1.885, 2.119] y el de R^2 es [0.584, 0.629]. La mejora del 28 % frente al baseline es consistente, pero el WAPE de 70 % recuerda que la demanda intermitente es difícil: el modelo acierta la tendencia, no cada pico. El sesgo medio es −0.049 unidades/día, ligeramente negativo, sin señal de sobrepredicción sistemática.

LECTURA DE NEGOCIO

Con MAE ≈ 0.99 unidades/día, un gerente puede dimensionar reposición y stock de seguridad por producto. El modelo es más útil para ordenar prioridades y detectar series de alta rotación que para predecir un día exacto. El error relativo alto en series con muchos ceros obliga a complementar con reglas de negocio y revisión humana.

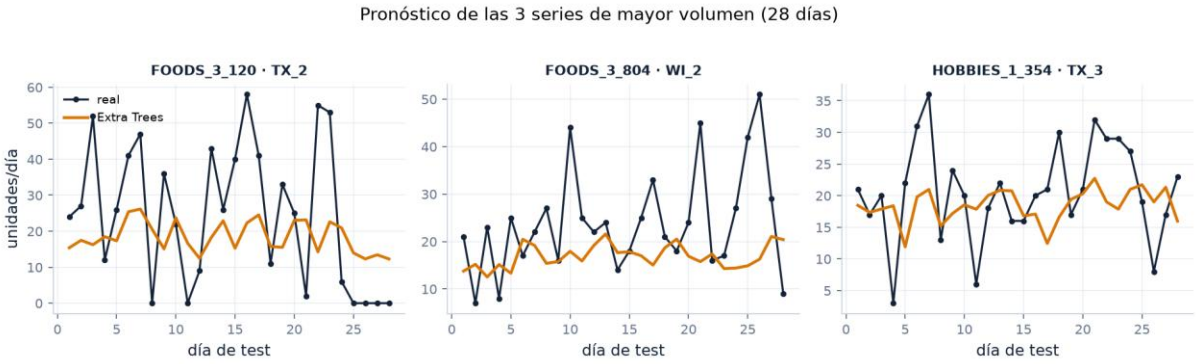


Figura 1. **Pronóstico de las tres series de mayor volumen en los 28 días de test. La línea oscura es la venta real y la ámbar la predicción.**

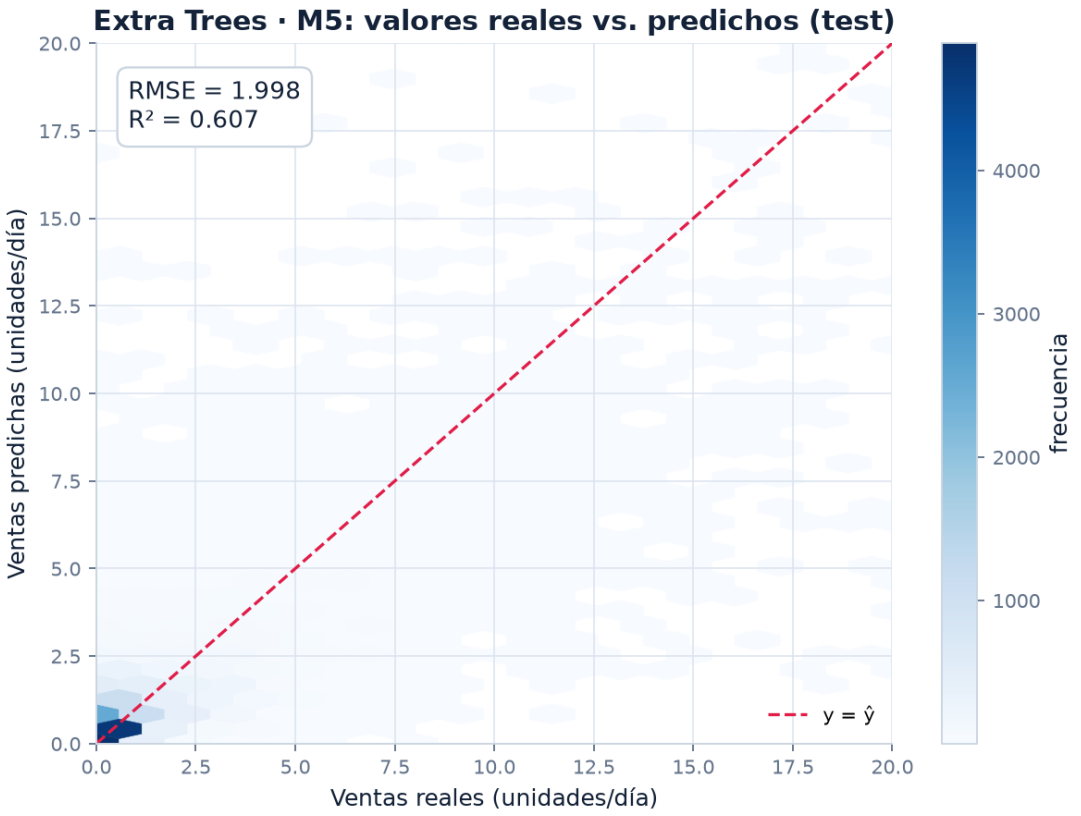


Figura 2. **Real vs. predicho en test. La nube se concentra en ventas bajas, típicas de demanda intermitente; los puntos de alta demanda se acercan a la diagonal.**

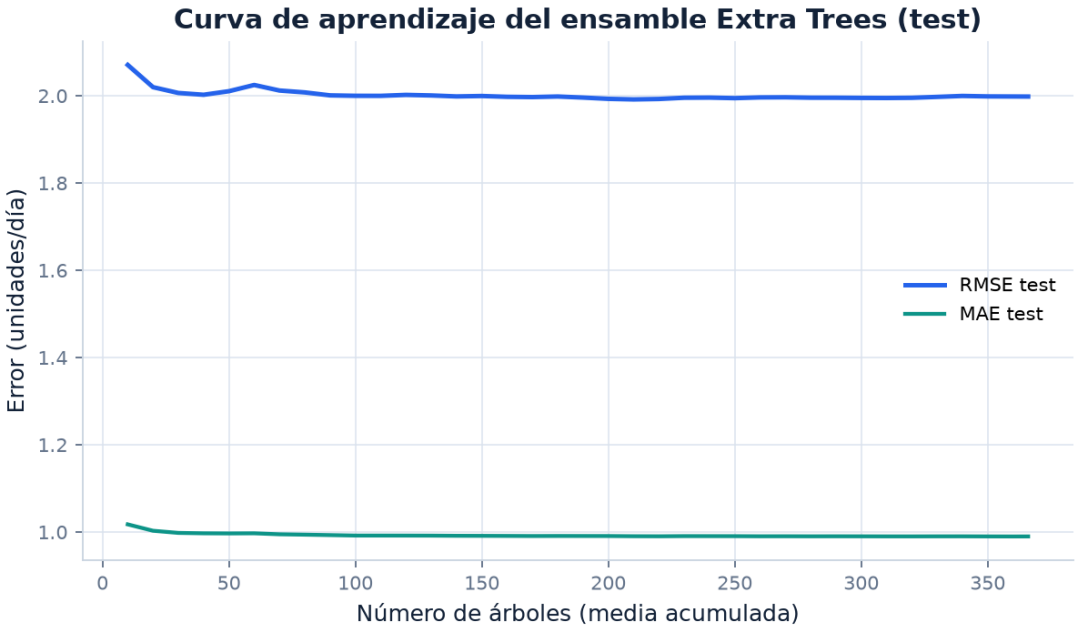


Figura 3. **Curva de aprendizaje sobre el test completo (22 400 filas): el error cae rápido y se estabiliza alrededor de 2.0 unidades/día. La curva no usa el conjunto de validación que participó del entrenamiento final.**

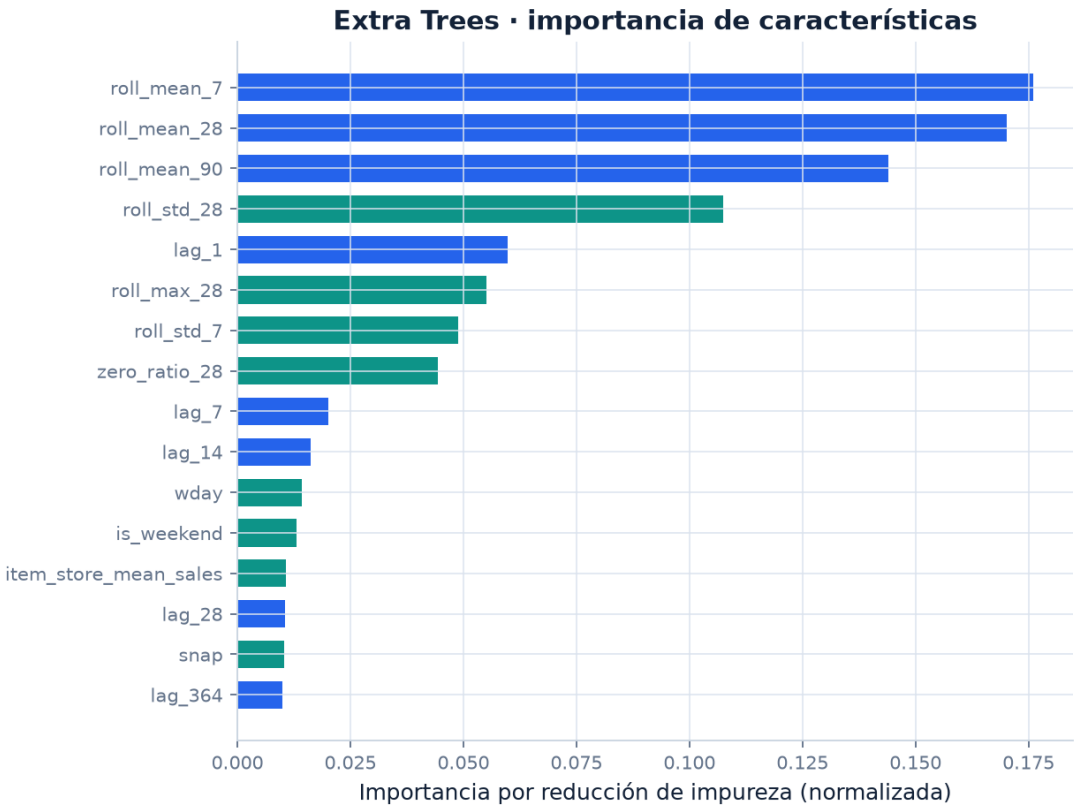


Figura 4. **Importancia por impureza. Los lags y las medias móviles dominan; el precio y el calendario aportan ajustes finos.**

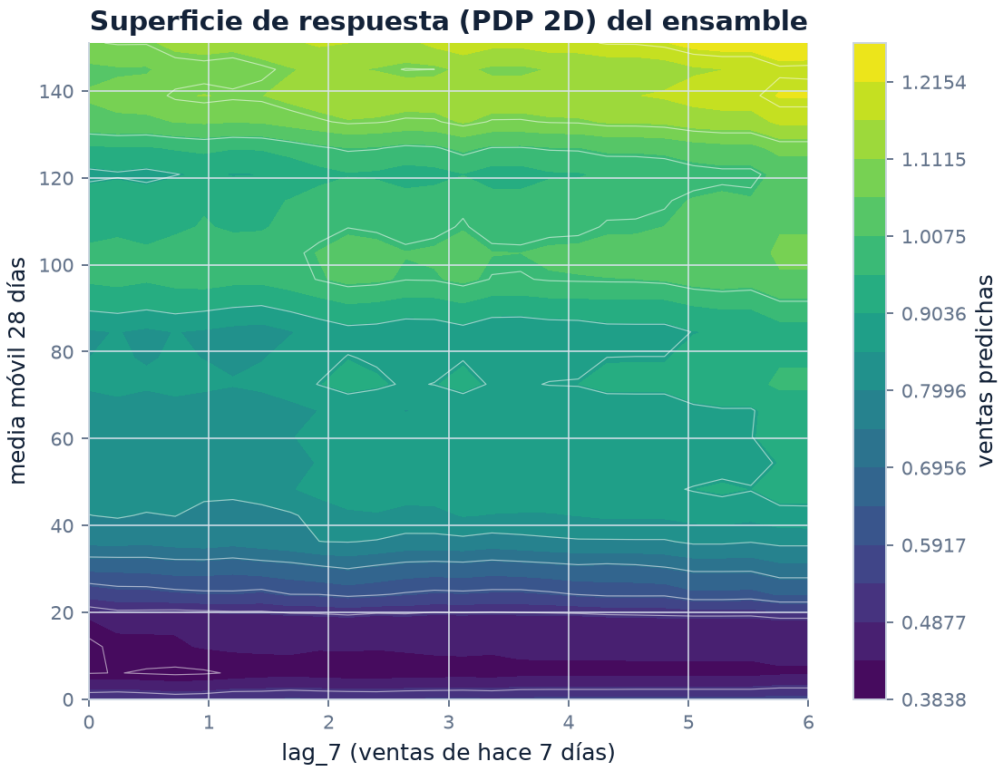


Figura 5. **Superficie de respuesta parcial (PDP) sobre lag_7 y la media móvil de 28 días. La demanda es casi monótona en ambas variables.**

4.2 AdaBoost · CDC BRFSS

Búsqueda y configuración

La búsqueda eligió **n = 370, learning rate = 0.1, profundidad = 3 y min_samples_leaf = 20**, con AUC de validación cruzada 0.822. El umbral operativo se fijó en 0.38 sobre una partición interna del 15 % del entrenamiento. La Tabla 4 compara el umbral estándar 0.50 con el umbral operativo.

Métrica	Umbral 0.50	Umbral 0.38	Δ
Accuracy	0.747	0.727	-0.020
Precisión	0.737	0.670	-0.067
Recall	0.780	0.912	0.132
F1	0.758	0.773	0.015
Especificidad	0.712	0.536	-0.176
Brier	0.178	0.178	0.000

Tabla 4. Compromiso entre umbrales en test. Bajar el umbral prioriza recall (no dejar casos sin detectar) a costa de precisión y especificidad.

El AUC es 0.821 y el average precision 0.799; el intervalo bootstrap del AUC es [0.815, 0.828] y el del F1 es [0.766, 0.779]. La ganancia de recall (0.780 → 0.912) es grande y consistente; la pérdida de precisión (0.737 → 0.670) es el precio de derivar más personas a confirmación. En cribado, ese intercambio suele ser aceptable si la prueba de confirmación es barata y el costo de un falso negativo es alto.

CALIBRACIÓN

El Brier score es 0.178 y la curva de calibración muestra desviaciones en la zona alta. Las probabilidades sirven para ordenar riesgo, pero no para decir a una persona “tiene 63 % de probabilidad de diabetes”. Antes de un uso clínico se requiere recalibración (Platt o isotónica) y validación externa.

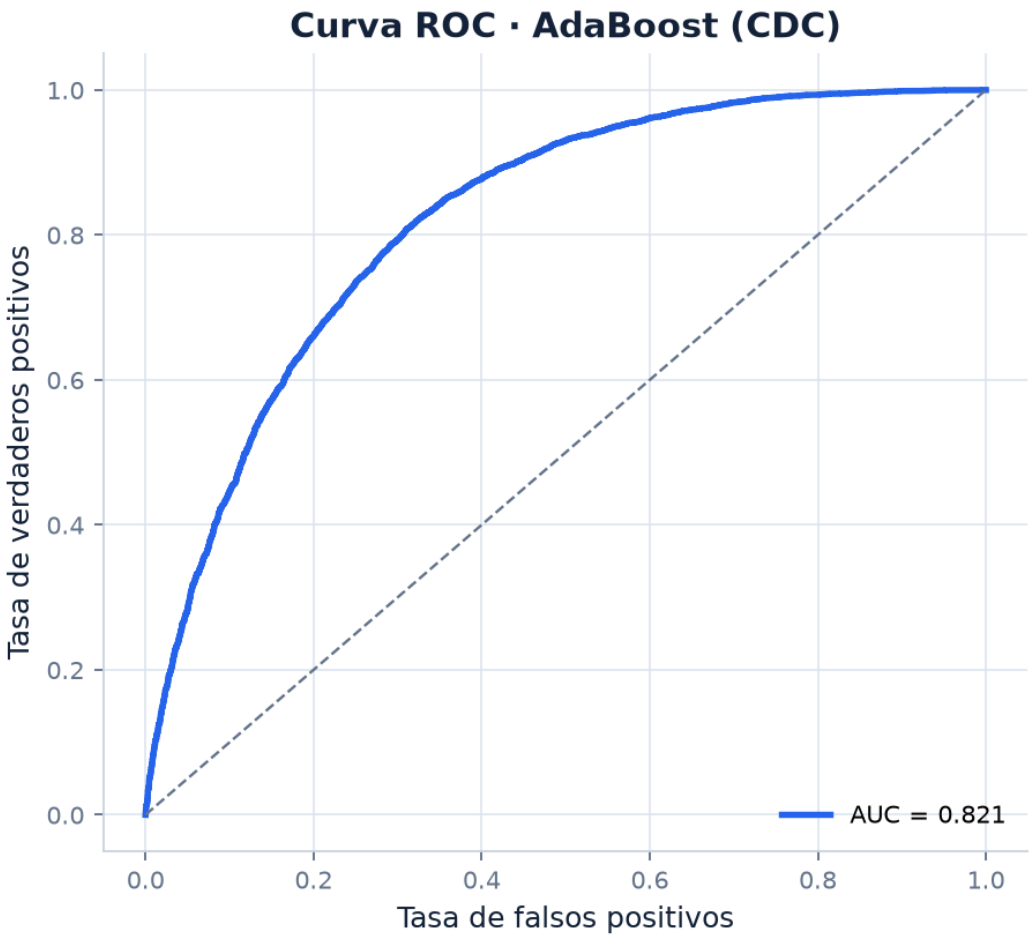


Figura 6. **Curva ROC en test. Un AUC de 0.821 indica buena separación entre personas con y sin diabetes.**

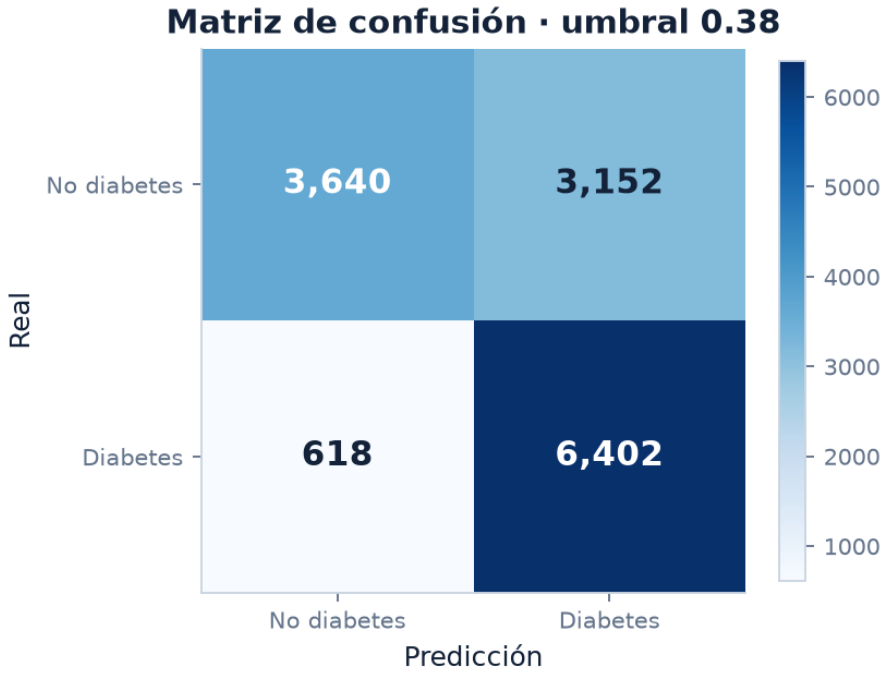


Figura 7. **Matriz de confusión con umbral 0.38. Se prioriza no dejar casos positivos sin detectar; los falsos positivos se derivan a confirmación.**

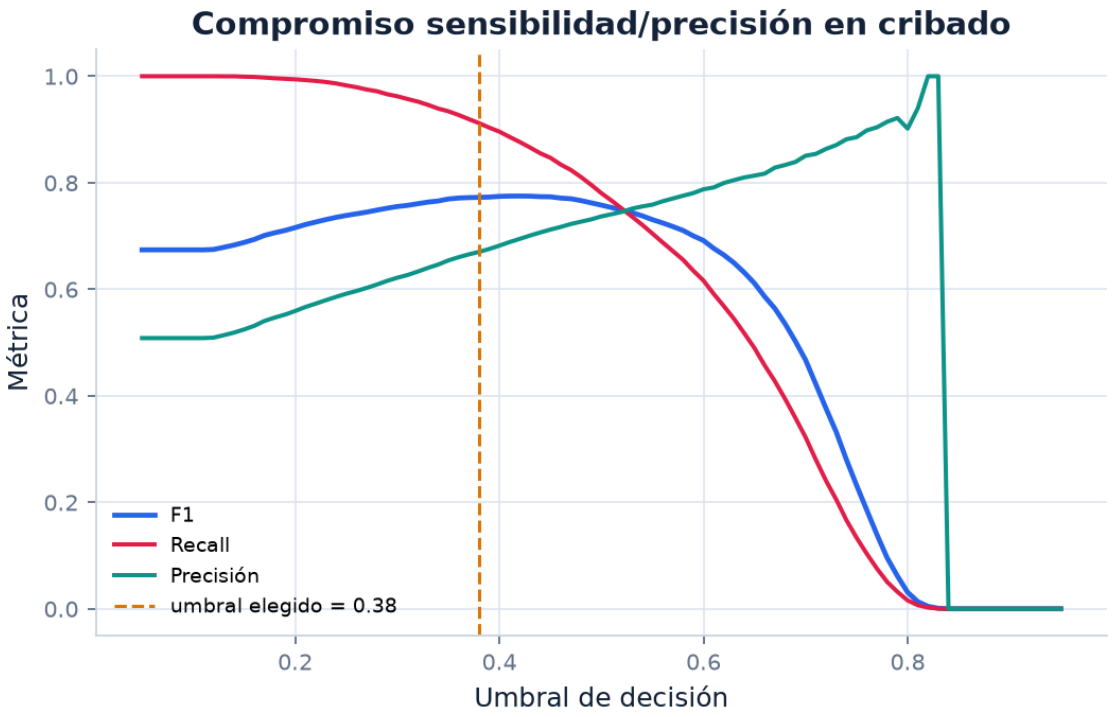


Figura 8. **Barrido de umbral. Bajar de 0.50 a 0.38 eleva el recall de 0.780 a 0.912 a costa de precisión.**

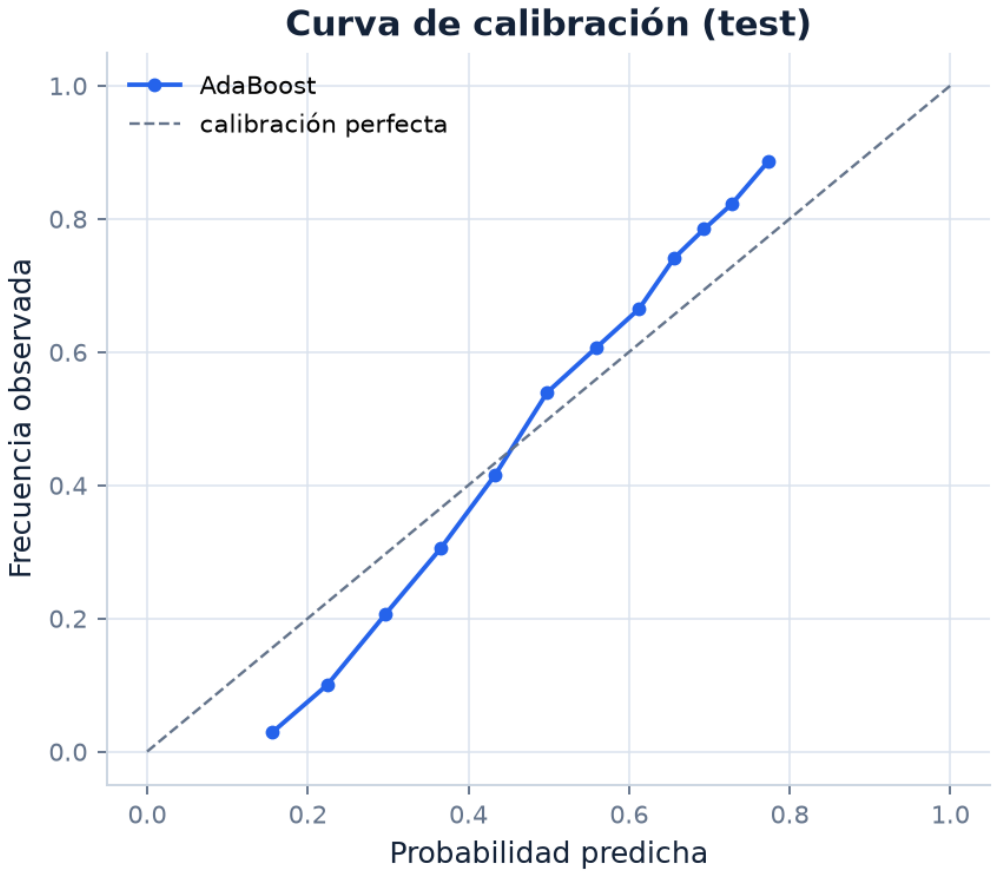


Figura 9. **Calibración de las probabilidades. La curva se aleja de la diagonal en los extremos; se recomienda recalibrar antes de comunicar riesgos individuales.**

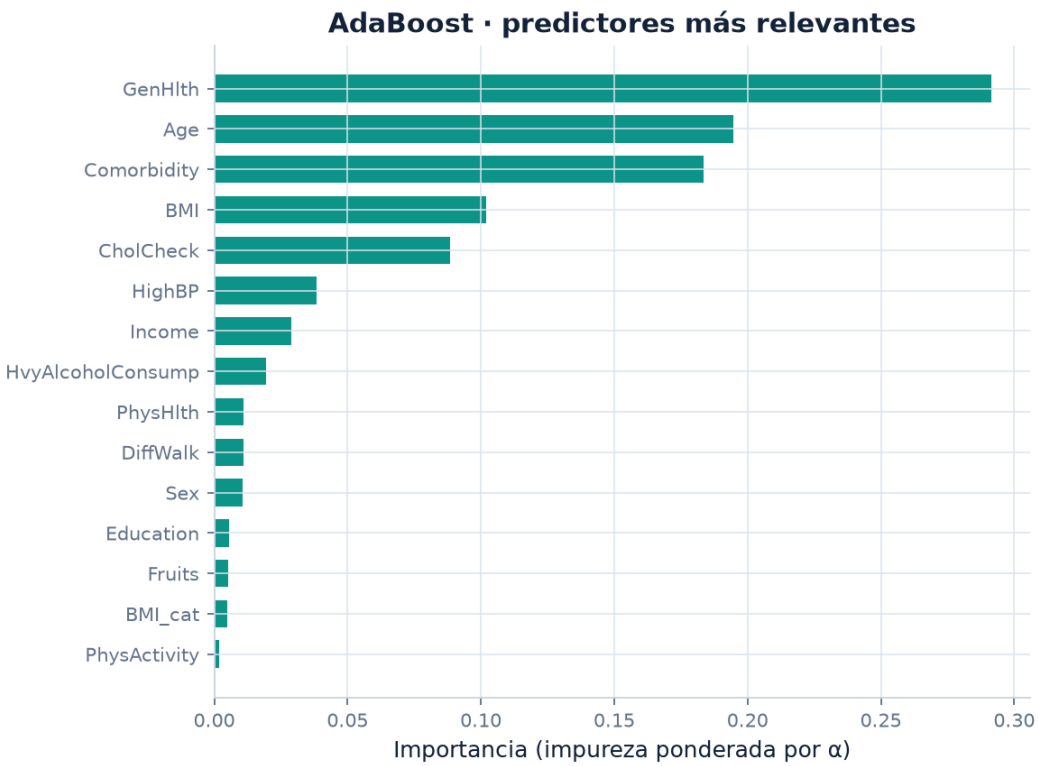


Figura 10. **Importancia de variables. Salud general, edad, comorbilidad y BMI concentran la señal; los hábitos aportan información incremental.**

4.3 CatBoost · UCI Student Dropout

Búsqueda y configuración

La mejor configuración fue **depth = 5, lr = 0.1, l2 = 5, iteraciones = 350, random_strength = 1, bagging_temperature = 0, auto_class_weights = Balanced**, con macro-F1 de validación cruzada 0.726. El early stopping usó 172 árboles de los 350 configurados. La Tabla 5 reporta las métricas globales y por clase.

Clase	Precisión	Recall	F1	Soporte
Dropout	0.804	0.694	0.745	284
Enrolled	0.418	0.629	0.503	159
Graduate	0.873	0.792	0.830	442

Tabla 5. Métricas por clase en test (885 estudiantes). Macro-F1 = 0.693; weighted-F1 = 0.744; accuracy = 0.731; balanced accuracy = 0.705.

El AUC-OvR es 0.886 y el log-loss 0.620. La clase Enrolled concentra la dificultad: su precisión (0.418) es baja porque comparte frontera con Dropout y Graduate. El modelo detecta bien Dropout (recall 0.694; F1 0.745) y Graduate (recall 0.792; F1 0.830), que son las decisiones de mayor consecuencia operativa. Los intervalos bootstrap son [0.703, 0.757] para accuracy y [0.664, 0.722] para macro-F1.

LECTURA DE NEGOCIO

El modelo no debe etiquetar a un estudiante como “abandono” de forma automática. Su valor es priorizar tutorías y revisar casos con un equipo humano. La clase Enrolled es una señal de que el sistema debe mirar trayectorias, no solo un snapshot: un estudiante en curso puede graduarse o abandonar con cambios pequeños en las variables.

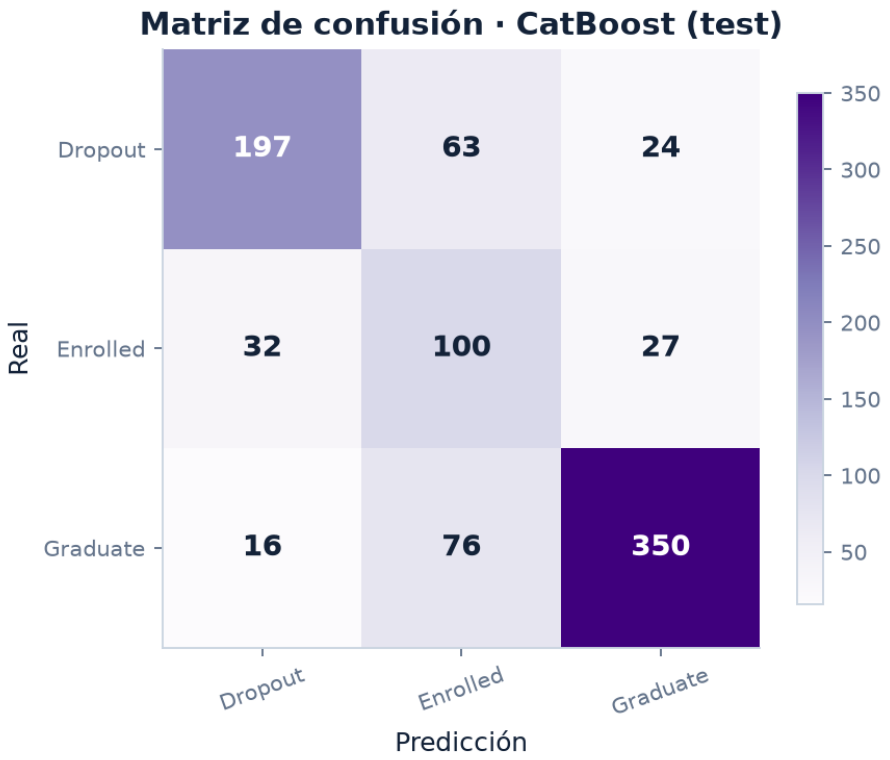


Figura 11. **Matriz de confusión 3×3. La clase Enrolled es la más difícil: comparte frontera con Dropout y Graduate.**

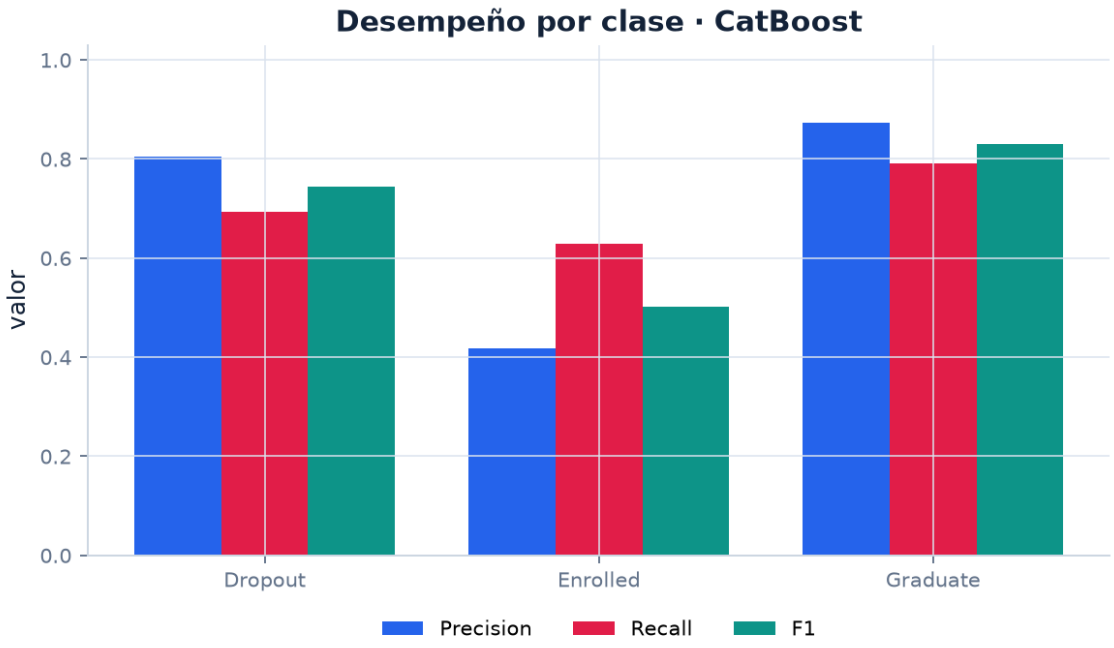


Figura 12. **Precisión, recall y F1 por clase. Dropout y Graduate superan 0.74 de F1; Enrolled queda en 0.50.**

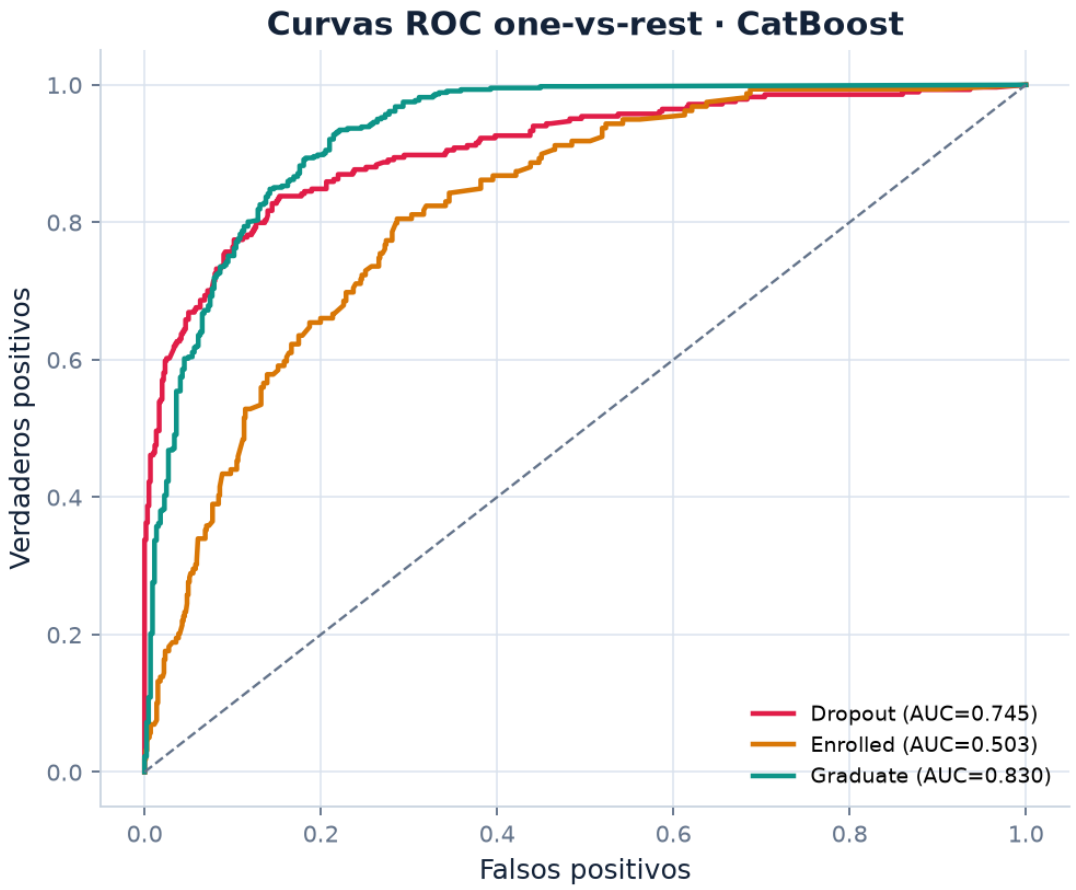


Figura 13. **Curvas ROC one-vs-rest. Graduate es la clase más separable; Enrolled, la más ambigua.**

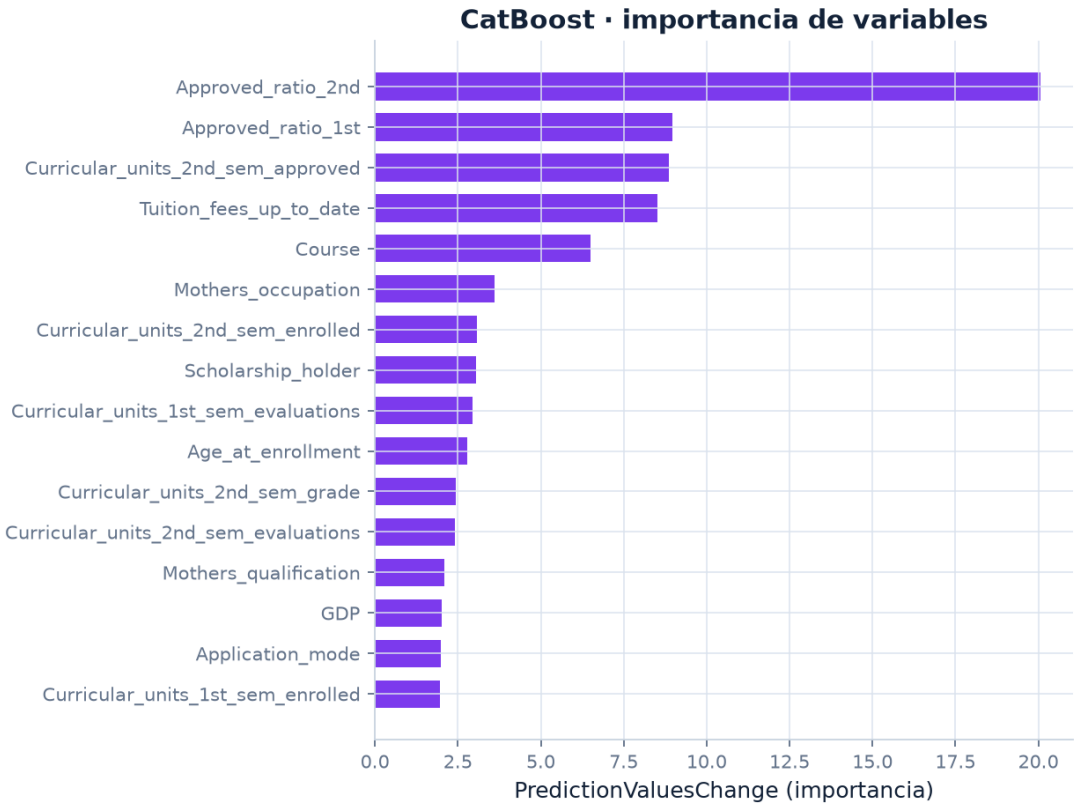


Figura 14. **Importancia de variables. Las unidades aprobadas del segundo semestre, la matrícula al día y el rendimiento previo dominan la predicción.**

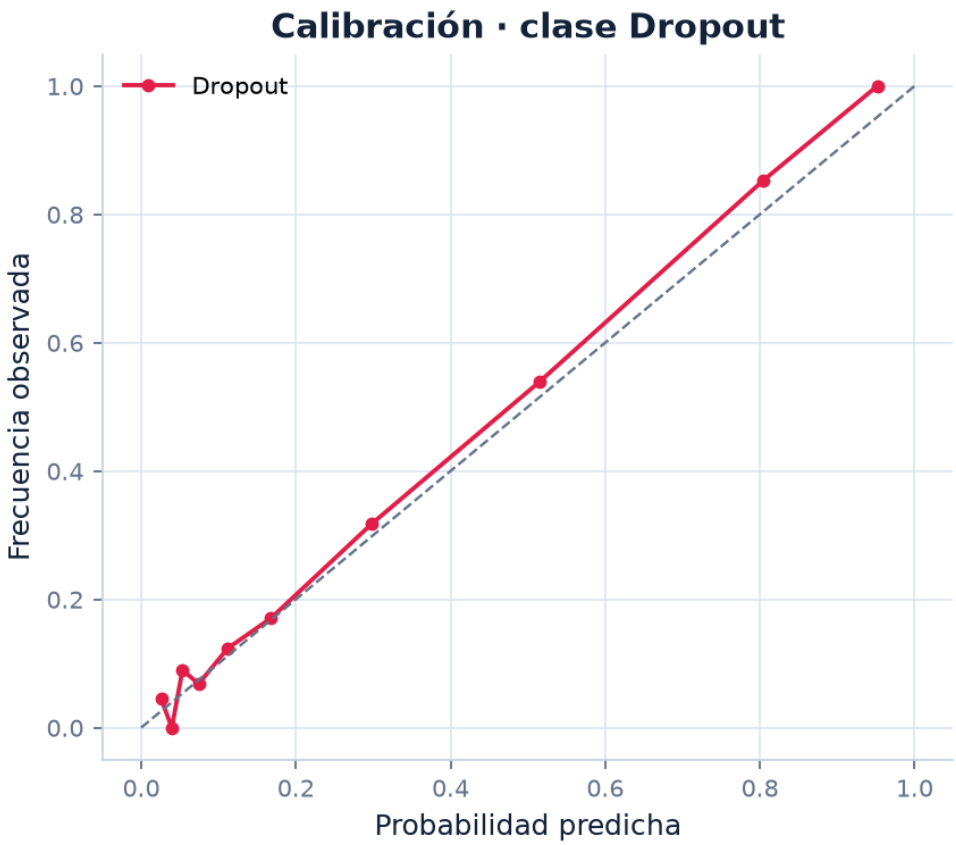


Figura 15. **Calibración de la probabilidad de Dropout. Es útil para priorizar tutorías, no para decisiones automáticas.**

4.4 Comparación entre los tres ensambles

La Tabla 6 resume el desempeño y el costo. La comparación debe leerse con cautela: las tareas, los tamaños y las métricas no son intercambiables. Un RMSE de 2.0 unidades/día no es "mejor" que un AUC de 0.82; son respuestas a preguntas distintas.

Modelo	Tarea	Test	Métrica principal	Métrica secundaria	Modelo
Extra Trees	Regresión	22 400	RMSE 1.998	R^2 0.607	2.9 GB
AdaBoost	Binaria	13 812	AUC 0.821	F1 0.758	0.57 MB
CatBoost	Multiclase	885	macro-F1 0.693	AUC-OvR 0.886	0.28 MB

Tabla 6. Comparación de desempeño y tamaño de modelo. Los tamaños son los joblib principales; los modelos demo del navegador son más pequeños.

En términos de esfuerzo de ajuste, CatBoost fue el más costoso (búsqueda y entrenamiento completos en ~16 minutos), seguido de Extra Trees (~7 minutos) y AdaBoost (~1.7 minutos). En inferencia interactiva, los tres responden en milisegundos; la diferencia práctica está en el tamaño y en la necesidad de preprocesamiento. AdaBoost y CatBoost ofrecen probabilidades o scores; Extra Trees entrega una predicción continua y una banda de incertidumbre entre árboles. La Figura 16 evidencia además que la búsqueda de Extra Trees se estabiliza en una meseta de RMSE, por lo que el modelo no exige un ajuste fino para rendir de forma competitiva.

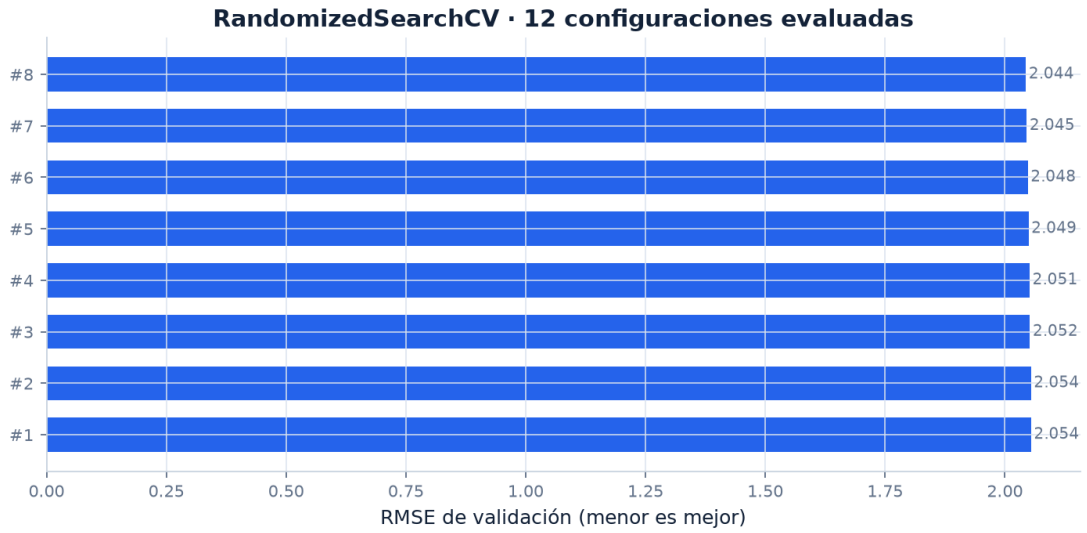


Figura 16. **Las 12 configuraciones de RandomizedSearchCV para Extra Trees. La mejor combina 366 árboles, profundidad 36 y 70 % de variables por split.**

La lectura integrada es que **no hay un ganador único**. Extra Trees es la opción natural para regresión tabular de alta dimensionalidad con ruido; AdaBoost es un baseline fuerte, barato e interpretable para clasificación binaria; CatBoost es la mejor opción cuando hay categóricas de alta cardinalidad y se necesita un modelo compacto y preciso. La sección 5 traduce esa conclusión a criterios de aplicación.

5. Aplicaciones y análisis crítico

Un modelo útil no es solo el que obtiene una métrica alta, sino el que mejora una decisión sin trasladar riesgos invisibles al usuario. Esta sección analiza cada caso desde esa perspectiva: qué decisión habilita, qué supuestos exige, qué puede fallar y qué controles humanos y técnicos necesita.

5.1 Retail: inventario con demanda intermitente

En M5, la decisión es cuánto reponer y cómo priorizar el surtido. El modelo entrega una predicción diaria por producto-tienda; el MAE de ~1 unidad/día es accionable a nivel de categoría, pero no debe usarse como orden de compra automática. La demanda tiene 59.75 % de ceros: un error absoluto de 1 unidad puede ser enorme en una serie que vende 0.2 unidades/día y pequeño en una que vende 20. Por eso se reporta WAPE y se recomienda segmentar por volumen.

- **Valor:** priorización de reposición, detección de series de alta rotación y planificación de seguridad de stock. El modelo reduce el RMSE un 28 % frente a lag_7.
- **Límite:** no extrapola tendencias ni promociones fuera del rango histórico; suaviza picos. La muestra de 800 series no cubre las 30 490 del dataset completo.
- **Control:** comparar contra un baseline estacional, monitorear deriva por tienda/departamento y mantener un override humano para eventos y quiebres de stock.
- **Ética/operación:** una mala predicción puede generar faltantes o exceso de inventario; el costo asimétrico debe incorporarse en la función de decisión, no solo en la métrica.

La implementación razonable es un **sistema de recomendación**, no un piloto automático: el modelo sugiere, el gerente ajusta y el sistema registra el resultado para reentrenar. La validación debe ser temporal y por tienda; una validación aleatoria mezclaría días futuros con pasados y sobreestimaría el desempeño.

5.2 Salud pública: cribado con recall alto

En BRFSS, la decisión es a quién invitar a una confirmación diagnóstica. El costo de un falso negativo es alto; el de un falso positivo es una prueba adicional y ansiedad. Por eso el umbral 0.38 es una decisión de política, no una propiedad del modelo: aumenta el recall a 0.912 y reduce la precisión a 0.670. El Brier de 0.178 y la curva de calibración muestran que las probabilidades no son clínicamente exactas; el modelo debe usarse como ranking de riesgo.

- **Valor:** cribado poblacional de bajo costo, con un modelo ligero (0.57 MB) y entrenamiento en segundos; permite priorizar recursos limitados.
- **Límite:** la encuesta es autodeclarada; el balanceo 50/50 no representa la prevalencia real; el modelo no incorpora historia clínica ni biomarcadores.
- **Equidad:** auditar recall y precisión por edad, sexo, ingreso y educación. Un modelo con buen promedio puede fallar en subgrupos pequeños.
- **Gobernanza:** no es diagnóstico; requiere confirmación clínica, explicación al paciente y revisión de sesgos antes de cualquier despliegue.

5.3 Educación: alerta temprana de abandono

En UCI, la decisión es qué estudiantes requieren acompañamiento. CatBoost detecta bien Dropout y Graduate, pero confunde Enrolled; además, el dataset es un snapshot con variables de dos semestres, no una secuencia temporal completa. Un despliegue real necesita validación temporal (predecir cohortes futuras), privacidad y un protocolo de intervención.

- **Valor:** prioriza tutorías y permite asignar recursos a estudiantes con mayor riesgo; la importancia de variables (aprobadas en 2.º semestre, matrícula al día) es accionable.
- **Límite:** 4 424 filas y 885 en test; la clase Enrolled es ambigua; el modelo puede perpetuar sesgos si se entrena con decisiones pasadas.
- **Ética:** una etiqueta de "abandono" puede estigmatizar. Debe comunicarse como probabilidad de necesitar apoyo, con explicaciones y derecho a revisión.
- **Control:** validación por cohorte, monitoreo de deriva y auditoría de equidad por programa, género y condiciones socioeconómicas.

5.4 Guía de selección

Situación	Modelo recomendado	Por qué
Regresión tabular, ruido, muchos outliers, necesidad de paralelizar	Extra Trees	Bagging extremo, bajo costo de tuning, robusto y rápido; no extrapola.
Clasificación binaria, baseline fuerte, interpretabilidad por rondas	AdaBoost	Ligero, secuencial, buena curva ROC; sensible a ruido y calibración imperfecta.
Categóricas de alta cardinalidad, multiclase, modelo compacto	CatBoost	Target statistics ordenadas, oblivious trees, buen desempeño con pocos datos.
Necesidad de probabilidades calibradas	Recalibrar cualquiera	Ninguno está calibrado por defecto; usar Platt/isotónica con validación externa.

Tabla 7. Guía práctica de selección. La decisión final depende de la métrica de negocio, el costo del error y los requisitos de explicabilidad.

6. Conclusiones

Los tres ensambles confirman una idea central: la elección del modelo es inseparable de la estructura del problema y del costo de equivocarse. Extra Trees aprovecha la aleatoriedad para estabilizar una regresión difícil y reduce el error un 28 % frente a un baseline estacional, con un MAE cercano a una unidad/día. AdaBoost demuestra que un modelo pequeño y rápido puede separar casos de diabetes con AUC 0.821; su umbral operativo convierte una métrica en una política de cribado. CatBoost muestra que las categóricas nativas y el boosting ordenado permiten un modelo compacto y preciso en un dataset pequeño y multiclase, aunque la clase Enrolled revela el límite de la definición del target.

La comparación no produce un ganador absoluto. Produce un **mapa de decisión**: Extra Trees para regresión tabular ruidosa y paralelizable; AdaBoost para clasificación binaria con necesidad de recall y bajo costo; CatBoost para categóricas de alta cardinalidad y multiclase. En los tres casos, la validación honesta, la separación de test, el reporte de intervalos y la auditoría de sesgos son tan importantes como el algoritmo.

Quedan límites claros: la muestra de M5 no cubre todas las series; el umbral de AdaBoost se eligió en una partición pequeña; CatBoost tiene solo 885 estudiantes en test; las probabilidades no están calibradas para uso clínico. La reproducibilidad está garantizada por el repositorio, las semillas fijas, los scripts de descarga y la verificación de paridad entre Python y JavaScript.

6.1 Trabajo futuro

- Validación temporal por cohorte y por tienda, con ventanas móviles y monitoreo de deriva.
- Recalibración de AdaBoost y CatBoost con Platt/isotónica y validación externa.
- Modelos jerárquicos para M5 (agregación por tienda/departamento) y funciones de pérdida asimétricas.
- Análisis de equidad por subgrupos y pruebas de estrés con datos sintéticos.
- Despliegue del dashboard como GitHub Pages y API de inferencia con límites de uso.

7. Referencias

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD*, 785–794.

Dorogush, A. V., Ershov, V., & Gulin, A. (2018). CatBoost: Gradient boosting with categorical features support. *arXiv:1810.11363*.

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences, 55*(1), 119–139.

Friedman, J., Hastie, T., & Tibshirani, R. (2000). Additive logistic regression: A statistical view of boosting. *The Annals of Statistics, 28*(2), 337–407.

Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning, 63*(1), 3–42.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning* (2nd ed.). Springer.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting, 38*(4), 1346–1364.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. *Advances in Neural Information Processing Systems, 31*.

Realinho, V., Machado, J., Baptista, L., & Martins, M. V. (2022). Predicting student dropout and academic success. *Data, 7*(11), 146.

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), e0118432.

Schapire, R. E., & Singer, Y. (1999). Improved boosting algorithms using confidence-rated predictions. *Machine Learning, 37*(3), 297–336.

Centers for Disease Control and Prevention. (2015). *Behavioral Risk Factor Surveillance System survey data*. U.S. Department of Health and Human Services.

UCI Machine Learning Repository. (2021). *Predict students' dropout and academic success* [Data set]. https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success

Walmart. (2022). *M5 forecasting — accuracy* [Data set]. Kaggle. https://www.kaggle.com/competitions/m5-forecasting-accuracy

A. Anexo A. Grillas de hiperparámetros

La Tabla A1 resume los espacios explorados. Los valores se implementaron con `scipy.stats` y listas discretas; el número de iteraciones de búsqueda se limitó para que el experimento fuera reproducible en un equipo personal.

Modelo	Espacio	Iteraciones	Validación
Extra Trees	n_estimators 150–400; max_depth 16–40; max_features 0.5/0.7/0.9; min_samples_leaf 2–24; min_samples_split 2–20; bootstrap True/False	12	1 split temporal (120k train / 22.4k val)
AdaBoost	n_estimators 100–420; learning_rate 0.01–0.4; depth 1–4; min_samples_leaf 1/5/10/20	14	StratifiedKFold(3)
CatBoost	iterations 250–650; depth 3–8; lr 0.02–0.10; l2 1–10; random_strength 0–2; bagging 0–1	14	StratifiedKFold(3)

Tabla A1. Espacios de búsqueda. Los mejores puntos se muestran en la Tabla 2.

B. Anexo B. Reproducibilidad y estructura

El repositorio se organiza así:

```
arboles-ensambles-equipo2/
├── docs/                # dashboard estático (GitHub Pages)
│   ├── index.html
│   ├── assets/{css,js}
│   ├── data/dashboard.json
│   └── models/*.json
├── src/                 # pipeline Python
│   ├── config.py
│   ├── data_m5.py · data_cdc.py · data_uci.py
│   ├── train_extra_trees.py
│   ├── train_adaboost.py
│   ├── train_catboost.py
│   └── export_web.py
├── artifacts/{figures,tables,models}
├── reports/             # DOCX, PDF, ecuaciones
├── scripts/download_data.py
├── run_all.py
└── requirements.txt
```

Comandos principales:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python scripts/download_data.py
python run_all.py
python -m src.make_report
powershell -ExecutionPolicy Bypass -File scripts/convert_pdf.ps1
```

ENTORNO DE EJECUCIÓN

Experimentos ejecutados en Windows con Python 3.14, scikit-learn 1.9, CatBoost 1.2.10, NumPy 2.5 y pandas 3.0. Los tiempos de pared fueron 429.9 s para Extra Trees, 99.0 s para AdaBoost y 957.7 s para CatBoost, incluyendo búsqueda, entrenamiento, figuras y exportación. No se usó GPU; son un orden de magnitud, no un benchmark.

Los datos crudos no se versionan por tamaño; el script de descarga reconstruye `data/raw/`. Los modelos principales se regeneran con `run_all.py`; el joblib de Extra Trees (2.9 GB) queda fuera de Git y se conserva solo localmente. El dashboard funciona con cualquier servidor estático (`python -m http.server` en `docs/`).

C. Anexo C. Declaración de uso responsable

Este trabajo es académico y reproducible. Los modelos no están validados para uso clínico, financiero o académico real. Cualquier despliegue debe someterse a revisión ética, validación externa, monitoreo de deriva y auditoría de equidad. Las decisiones que afectan a personas deben mantener revisión humana y mecanismos de apelación. Los datasets son públicos y anonimizados; aun así, deben respetarse las licencias y las condiciones de uso de cada fuente.